

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5

10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5
----	--	---------------	---

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1) Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

ALGORITHM Query_Processing(SQL_Query)

INPUT:

SQL_Query

OUTPUT:

Query_Result

BEGIN

// Step 1: SQL Input

Receive SQL_Query from the user

// Step 2: Parsing and Validation

Tokens $\leftarrow$ Lexical_Analysis(SQL_Query)

Parse_Tree $\leftarrow$ Syntax_Analysis(Tokens)

IF Parse_Tree contains syntax error THEN

Display "Syntax Error"

STOP

END IF

Validated_Tree $\leftarrow$ Semantic_Analysis(Parse_Tree)

IF Validated_Tree contains semantic error THEN

Display "Semantic Error"

STOP

END IF

// Step 3: Query Translation

Relational_Algebra $\leftarrow$ Translate_To_Relational_Algebra(Validated_Tree)

// Example:

// SELECT name FROM Student WHERE age > 18

// ↓

// PROJECT name (SELECT age > 18 (Student))

// Step 4: Query Optimization

Candidate_Plans $\leftarrow$ Generate_Execution_Plans(Relational_Algebra)

```

FOR each Plan in Candidate_Plans DO
    Cost[Plan] ← Estimate_Cost(
        Disk_I_O,
        CPU_Cost,
        Memory_Usage,
        Network_Cost
    )
END FOR

Optimal_Plan ← Select_Plan_With_Minimum_Cost(Cost)

```

```

// Step 5: Query Execution
Initialize Execution_Engine

```

```

Result ← Execute(Optimal_Plan)

WHILE more_operations_exist(Optimal_Plan) DO
    Operation ← Get_Next_Operation(Optimal_Plan)

    IF Operation requires table data THEN
        Data ← Access_Database(Storage_Manager)
    END IF

    IF Operation is SELECT THEN
        Result ← Apply_Selection(Data)

    ELSE IF Operation is JOIN THEN
        Result ← Apply_Join(Data)

    ELSE IF Operation is PROJECT THEN
        Result ← Apply_Projection(Data)

    ELSE IF Operation is SORT THEN
        Result ← Apply_Sorting(Data)

    ELSE IF Operation is GROUP_BY THEN
        Result ← Apply_Grouping(Data)

    ELSE IF Operation is AGGREGATE THEN
        Result ← Apply_Aggregate_Function(Data)
    END IF
END WHILE

```

```

// Step 6: Return Result
Format(Result)

Display Result to the user

```

```

END

```

2) Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

ALGORITHM Cost_Based_Join_Optimization

INPUT:

Relations = {R1, R2, ..., Rn}
Join_Conditions
Statistics for each relation
- Number of tuples
- Number of blocks
- Available indexes
- Selectivity

OUTPUT:

Optimal_Join_Order

BEGIN

// Step 1: Generate possible join orderings
Join_Orders $\leftarrow$ Generate_All_Join_Orderings(Relations)

Best_Cost $\leftarrow$ INFINITY
Best_Order $\leftarrow$ NULL

// Step 2: Evaluate each join ordering
FOR each Order IN Join_Orders DO

Current_Result $\leftarrow$ First_Relation(Order)
Total_Cost $\leftarrow$ 0

FOR each Next_Relation IN Remaining_Relations(Order) DO

// Estimate size of current intermediate result
Current_Blocks $\leftarrow$ Estimate_Blocks(Current_Result)

// Estimate size of next relation
Next_Blocks $\leftarrow$ Get_Block_Count(Next_Relation)

// Estimate join selectivity
Selectivity $\leftarrow$ Estimate_Join_Selectivity(
 Current_Result,
 Next_Relation,
 Join_Conditions
)

// Step 3: Estimate I/O cost
// Example: Block Nested-Loop Join
Join_Cost $\leftarrow$
 Current_Blocks +
 (Current_Blocks $\times$ Next_Blocks)

```

// Estimate size of new intermediate result
Result_Blocks ←
    Estimate_Join_Result_Size(
        Current_Result,
        Next_Relation,
        Selectivity
    )

// Add cost of this join
Total_Cost ← Total_Cost + Join_Cost

// Update intermediate result
Current_Result ← Create_Intermediate_Result(
    Current_Result,
    Next_Relation,
    Result_Blocks
)

```

END FOR

```

// Step 4: Compare with the best plan
IF Total_Cost < Best_Cost THEN
    Best_Cost ← Total_Cost
    Best_Order ← Order
END IF

```

END FOR

```

// Step 5: Return optimal join order
RETURN Best_Order, Best_Cost

```

END

3) Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Nested Loop Join – Pseudocode:

ALGORITHM Nested_Loop_Join(R, S, Join_Condition)

INPUT:

Relation R

Relation S

Join condition

OUTPUT:

Joined relation

BEGIN

Result $\leftarrow$ Empty

FOR each tuple r IN R DO

FOR each tuple s IN S DO

IF Join_Condition(r, s) = TRUE THEN

Add (r, s) to Result

END IF

END FOR

END FOR

RETURN Result

END

Block Nested Loop Join:

ALGORITHM Block_Nested_Loop_Join(R, S, M)

BEGIN

Result $\leftarrow$ Empty

FOR each block of R using (M - 2) buffer pages DO

Load block of R into memory

FOR each block of S DO

Load block of S into memory

FOR each tuple r in R_block DO

FOR each tuple s in S_block DO

IF Join_Condition(r, s) = TRUE THEN

Add (r, s) to Result

END IF

END FOR

END FOR

END FOR

END FOR

RETURN Result

END

4) Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

ALGORITHM Detect_Deadlock_WFG

INPUT:

Transactions = {T1, T2, ..., Tn}
Resource_Allocation
Waiting_Requests

OUTPUT:

Deadlock status and transactions involved

BEGIN

// Step 1: Create an empty Wait-For Graph
WFG $\leftarrow$ Empty_Graph

// Step 2: Add all transactions as nodes
FOR each transaction T IN Transactions DO
 Add_Node(WFG, T)
END FOR

// Step 3: Create edges based on waiting relationships
FOR each transaction Ti IN Transactions DO

 FOR each resource R requested by Ti DO

 IF R is currently held by transaction Tj THEN

 // Ti is waiting for Tj
 Add_Edge(WFG, Ti, Tj)

 END IF

 END FOR

END FOR

// Step 4: Detect cycle using DFS
FOR each transaction T IN Transactions DO
 Visited[T] $\leftarrow$ FALSE
 In_Stack[T] $\leftarrow$ FALSE
END FOR

FOR each transaction T IN Transactions DO

IF Visited[T] = FALSE THEN

IF DFS_Cycle_Detection(WFG, T) = TRUE THEN

Deadlock_Transactions $\leftarrow$ Find_Cycle(WFG, T)

Display "Deadlock Detected"

Display Deadlock_Transactions

RETURN TRUE

END IF

END IF

END FOR

Display "No Deadlock Detected"

RETURN FALSE

END

5) Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

ALGORITHM Basic_2PL(Transaction T)

INPUT:

Transaction T containing a sequence of read/write operations

OUTPUT:

Completed or aborted transaction

BEGIN

Lock_Set $\leftarrow$ Empty

Phase $\leftarrow$ GROWING

FOR each operation OP in T DO

// -----

// Growing Phase

// -----

IF OP requires READ access THEN

 IF T does not hold a lock on the required item THEN

 Request Shared_Lock(X)

 IF Shared_Lock(X) is available THEN

 Grant Shared_Lock(X)

 Add X to Lock_Set

 ELSE

 WAIT until Shared_Lock(X) becomes available

 Grant Shared_Lock(X)

 Add X to Lock_Set

 END IF

 END IF

 Perform READ(X)

ELSE IF OP requires WRITE access THEN

 IF T does not hold an Exclusive_Lock on X THEN

 Request Exclusive_Lock(X)

 IF Exclusive_Lock(X) is available THEN

 Grant Exclusive_Lock(X)

 Add X to Lock_Set

 ELSE

 WAIT until Exclusive_Lock(X) becomes available

 Grant Exclusive_Lock(X)

 Add X to Lock_Set

END IF

END IF

Perform WRITE(X)

END IF

// -----

// Begin Shrinking Phase

// -----

IF T decides to release a lock THEN

Phase $\leftarrow$ SHRINKING

Release_Lock(X)

Remove X from Lock_Set

// No new locks can be acquired

IF T requests a new lock THEN

WAIT or ABORT transaction

END IF

END IF

END FOR

// Release remaining locks

Phase $\leftarrow$ SHRINKING

FOR each X in Lock_Set DO

Release_Lock(X)

END FOR

COMMIT T

END

6) Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

ALGORITHM Thomas_Write_Rule(T, Operation, X)

INPUT:

Transaction T
Operation = READ or WRITE
Data item X

DATA:

TS(T) = Timestamp of transaction T
RTS(X) = Read timestamp of X
WTS(X) = Write timestamp of X

OUTPUT:

Operation result, WAIT, IGNORE, or ABORT

BEGIN

IF Operation = READ(X) THEN

// Read Timestamp Ordering Rule
IF TS(T) < WTS(X) THEN

// X was already written by a newer transaction
ABORT T
RETURN ABORT

ELSE

// Read is allowed
Perform READ(X)

RTS(X) $\leftarrow$ MAX(RTS(X), TS(T))

RETURN SUCCESS

END IF

ELSE IF Operation = WRITE(X) THEN

// Case 1: Write is too late because
// a newer transaction has already read X
IF TS(T) < RTS(X) THEN

ABORT T
RETURN ABORT

// Case 2: Write is obsolete
ELSE IF TS(T) < WTS(X) THEN

```
// Thomas' Write Rule
// Ignore the obsolete write
IGNORE WRITE(X)
```

```
RETURN IGNORE
```

```
// Case 3: Write is valid
ELSE
```

```
    Perform WRITE(X)
```

```
     $WTS(X) \leftarrow TS(T)$ 
```

```
    RETURN SUCCESS
```

```
END IF
```

```
END IF
```

```
END
```

7) Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

ALGORITHM ARIES_RECOVERY

INPUT:

Log
Database
Checkpoint information

OUTPUT:

Recovered and consistent Database

BEGIN

```
// =====  
// PHASE 1: ANALYSIS  
// =====
```

Read the most recent checkpoint

Initialize:

Transaction_Table $\leftarrow$ Empty
Dirty_Page_Table $\leftarrow$ Empty

FOR each log record L from checkpoint to end of Log DO

IF L represents TRANSACTION_START THEN

Add transaction L.TID to Transaction_Table
Set its status to ACTIVE

ELSE IF L represents UPDATE THEN

Add L.TID to Transaction_Table

IF L.PageID is not in Dirty_Page_Table THEN

Add L.PageID to Dirty_Page_Table
Dirty_Page_Table[L.PageID].recLSN $\leftarrow$ L.LSN
END IF

Update Transaction_Table[L.TID].lastLSN $\leftarrow$ L.LSN

ELSE IF L represents COMMIT THEN

Transaction_Table[L.TID].status $\leftarrow$ COMMITTED
Transaction_Table[L.TID].lastLSN $\leftarrow$ L.LSN

ELSE IF L represents ABORT THEN

Transaction_Table[L.TID].status ← ABORTING

Transaction_Table[L.TID].lastLSN ← L.LSN

ELSE IF L represents END THEN

Remove L.TID from Transaction_Table

ELSE IF L represents CLR THEN

Update Transaction_Table[L.TID].lastLSN ← L.LSN

END IF

END FOR

// Transactions still active/aborting after crash

FOR each transaction T in Transaction_Table DO

IF T.status = ACTIVE THEN

T.status ← ABORTING

END IF

END FOR

// =====

// PHASE 2: REDO

// =====

Redo_Start ← Smallest recLSN in Dirty_Page_Table

IF Redo_Start exists THEN

FOR each log record L from Redo_Start to end of Log DO

IF L is a REDOABLE operation THEN

Page ← Load_Page(L.PageID)

IF L.LSN > Page.PageLSN THEN

REDO L

Page.PageLSN ← L.LSN

Write_Page(Page)

END IF

END IF

END FOR

END IF


```
// =====  
// PHASE 3: UNDO  
// =====
```

Create Undo_List containing
 lastLSN of every transaction
 whose status is ABORTING

WHILE Undo_List is not empty DO

 L ← Get_Largest_LSN(Undo_List)

 IF L is an UPDATE record THEN

 // Undo the update
 Undo_Operation(L)

 // Write a Compensation Log Record
 CLR ← Create_CLR(L)

 Append CLR to Log

 Force_Log_To_Disk()

 END IF

 IF L.prevLSN exists THEN
 Add L.prevLSN to Undo_List

 ELSE
 // Transaction has been completely undone
 Create END record for L.TID
 Append END record to Log

 Remove L.TID from Transaction_Table
 END IF

END WHILE

```
// =====  
// RECOVERY COMPLETE  
// =====
```

Flush all required pages to disk

RETURN Recovered Database

END

8) Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

ALGORITHM Shadow_Paging_Recovery

INPUT:

Database
Transaction operations

OUTPUT:

Committed database or restored database

BEGIN

```
// =====  
// STEP 1: Initialize Shadow Page Table  
// =====
```

Shadow_Page_Table $\leftarrow$ Copy(Current_Page_Table)

// Shadow table is kept unchanged during execution

```
// =====  
// STEP 2: Create Current Page Table  
// =====
```

Current_Page_Table $\leftarrow$ Copy(Shadow_Page_Table)

Transaction_Status $\leftarrow$ ACTIVE

```
// =====  
// STEP 3: Execute Transaction  
// =====
```

FOR each operation OP in Transaction DO

IF OP is READ(Page P) THEN

Physical_Page $\leftarrow$ Current_Page_Table[P]
Read data from Physical_Page

ELSE IF OP is WRITE(Page P, Data) THEN

// Do not overwrite the original page

New_Page $\leftarrow$ Allocate_New_Page()

Copy contents of Current_Page_Table[P]
to New_Page

Apply Data modification to New_Page

Current_Page_Table[P] ← New_Page

END IF

END FOR

```
// =====  
// STEP 4: Commit  
// =====
```

IF Transaction completes successfully THEN

Flush all modified pages to disk

Flush Current_Page_Table to disk

// Atomic pointer switch

Database_Page_Table_Pointer ← Current_Page_Table

Transaction_Status ← COMMITTED

Delete Old Shadow_Page_Table

RETURN "Transaction Committed"

END IF

```
// =====  
// STEP 5: Failure / Crash Recovery  
// =====
```

IF System_Failure occurs THEN

// Discard changes made using current table

Discard Current_Page_Table

// Restore the stable database

Database_Page_Table_Pointer ← Shadow_Page_Table

Transaction_Status ← ABORTED

RETURN "Transaction Rolled Back"

END IF

END

9) Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

ALGORITHM Immediate_Update_Recovery

INPUT:

Log
Database

OUTPUT:

Recovered and consistent Database

BEGIN

```
// =====  
// STEP 1: Scan the Log  
// =====
```

Committed $\leftarrow$ Empty_Set
Uncommitted $\leftarrow$ Empty_Set

FOR each log record L in Log DO

IF L.type = START THEN
Add L.Transaction_ID to Uncommitted

ELSE IF L.type = COMMIT THEN
Add L.Transaction_ID to Committed
Remove L.Transaction_ID from Uncommitted

END IF

END FOR

```
// =====  
// STEP 2: REDO Committed Transactions  
// =====
```

FOR each log record L in Log from beginning to end DO

IF L.type = UPDATE AND
L.Transaction_ID is in Committed THEN

// Apply the new value
Write L.New_Value to L.Data_Item

END IF

END FOR

```
// =====  
// STEP 3: UNDO Uncommitted Transactions
```

```
// =====
```

```
FOR each log record L in Log from end to beginning DO
```

```
    IF L.type = UPDATE AND
```

```
        L.Transaction_ID is in Uncommitted THEN
```

```
        // Restore the old value
```

```
        Write L.Old_Value to L.Data_Item
```

```
    END IF
```

```
END FOR
```

```
// =====
```

```
// STEP 4: Finish Recovery
```

```
// =====
```

```
Flush all modified database pages to disk
```

```
RETURN "Recovery Completed"
```

```
END
```

10) Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

ALGORITHM Database_Connectivity_Manager

INPUT:

Database configuration
SQL Query
Query Parameters

OUTPUT:

Query Result or Error

BEGIN

```
// =====  
// STEP 1: Initialize Connection Pool  
// =====
```

```
Connection_Pool ← Create_Pool(  
    Database_URL,  
    Username,  
    Password,  
    Minimum_Connections,  
    Maximum_Connections  
)
```

```
// =====  
// STEP 2: Get Connection from Pool  
// =====
```

TRY

```
Connection ← Connection_Pool.Get_Connection()
```

```
IF Connection = NULL THEN  
    Display "No database connection available"  
    RETURN FAILURE  
END IF
```

```
// =====  
// STEP 3: Start Transaction  
// =====
```

```
Set AutoCommit(Connection, FALSE)
```

```
// =====  
// STEP 4: Execute Query  
// =====
```

```
Prepared_Query ← Prepare_Statement(  
    Connection,  
    SQL_Query  
)
```

```
Bind_Query_Parameters to Prepared_Query
```

```
Result ← Execute(Prepared_Query)
```

```
// =====  
// STEP 5: Commit Transaction  
// =====
```

```
Commit(Connection)
```

```
RETURN Result
```

```
// =====  
// STEP 6: Handle Exception  
// =====
```

```
CATCH Exception E
```

```
Display "Database Error: ", E.Message
```

```
IF Connection is active THEN  
    Rollback(Connection)  
END IF
```

```
RETURN FAILURE
```

```
// =====  
// STEP 7: Release Connection  
// =====
```

```
FINALLY
```

```
IF Prepared_Query exists THEN  
    Close(Prepared_Query)  
END IF
```

```
IF Connection exists THEN  
    Reset(Connection)  
    Connection_Pool.Return_Connection(Connection)  
END IF
```

```
END
```